MODUL 4 ADC DAN PWM

Apriyudha (H1H024010)
Asisten: Arga Aryanta Indrafata
Tanggal Percobaan: 28/04/2026
TK244005-Praktikum Sistem Mikrokontroler
Laboratorium Multimedia – Jurusan Informatika UNSOED

Abstrak

Praktikum ini bertujuan untuk memahami dan mengimplementasikan konsep Analog to Digital Converter (ADC) dan Pulse Width Modulation (PWM) pada mikrokontroler Arduino. ADC memungkinkan Arduino membaca sinyal analog dari sensor dan mengubahnya menjadi nilai digital, sedangkan PWM memungkinkan Arduino menghasilkan sinyal analog semu dari keluaran digital untuk mengontrol perangkat seperti servo motor dan LED. Dua percobaan dilakukan: pertama, pembacaan nilai potensiometer menggunakan fungsi analogRead() dan konversi ke sudut servo menggunakan fungsi map(); kedua, pembacaan nilai potensiometer dan konversi ke nilai PWM untuk mengatur kecerahan LED menggunakan fungsi analogWrite(). Hasil percobaan menunjukkan bahwa ADC berhasil membaca nilai analog dari potensiometer (rentang 0-1023) dan mengkonversinya ke sudut servo (0-180 derajat) dan nilai PWM LED (0-255). Servo motor bergerak sesuai perubahan posisi potensiometer dan LED merespons perubahan nilai PWM dengan variasi kecerahan yang sesuai.

Kata kunci:*ADC, PWM, Arduino, Servo Motor, analogRead, analogWrite*

1. Pendahuluan

Dalam pengembangan sistem embedded menggunakan Arduino, dua konsep penting yang sering digunakan adalah Analog to Digital Converter (ADC) dan Pulse Width Modulation (PWM). Keduanya memungkinkan Arduino untuk berinteraksi dengan dunia analog, baik dalam membaca sensor maupun mengontrol perangkat output. ADC dan PWM sering digunakan bersamaan dalam berbagai aplikasi, misalnya membaca nilai dari potensiometer (ADC) dan menggunakan nilai tersebut untuk mengatur kecerahan LED (PWM). Dengan kombinasi ini, Arduino dapat menerima input analog, memproses data, dan menghasilkan output yang responsif [1].

Pada sistem digital, keluaran mikrokontroler pada umumnya hanya memiliki dua kondisi logika, yaitu HIGH (1) dan LOW (0). Kondisi ini menyebabkan perangkat keluaran seperti LED secara alami hanya dapat dikendalikan dalam keadaan menyala atau mati. Namun, dalam banyak aplikasi sistem tertanam seperti pengaturan kecerahan lampu, pengendalian kecepatan motor DC, dan manajemen daya dibutuhkan kemampuan untuk mengatur tingkat keluaran secara bertahap, bukan sekadar dua kondisi ekstrem tersebut [2].

Praktikum ini dilakukan untuk memberikan pemahaman praktis mengenai konfigurasi dan penggunaan modul ADC dan PWM pada mikrokontroler Arduino. Percobaan pertama fokus pada penggunaan ADC untuk membaca nilai analog dari potensiometer dan mengkonversinya ke sudut servo motor. Percobaan kedua berfokus pada penggunaan PWM untuk mengatur kecerahan LED berdasarkan nilai analog yang dibaca dari potensiometer [3].

2. Studi Pustaka

2.1 Analog to Digital Converter (ADC)

Analog to Digital Converter (ADC) adalah modul pada mikrokontroler yang berfungsi mengubah sinyal analog menjadi nilai digital. Arduino Uno menggunakan mikrokontroler ATmega328P yang memiliki ADC built-in dengan resolusi 10-bit. Artinya, rentang nilai tegangan input (0-5V) akan dikonversi menjadi nilai digital antara 0 hingga 1023. Nilai 0 merepresentasikan tegangan 0V dan nilai 1023 merepresentasikan tegangan referensi (biasanya 5V) [1].

ATmega328P menggunakan arsitektur Successive Approximation Register (SAR) untuk melakukan konversi ADC. Proses konversi terdiri dari empat tahap: sample phase, hold phase, conversion phase, dan output phase. Seluruh proses memerlukan 13 siklus clock ADC. Dengan prescaler default 128 pada sistem clock 16 MHz, diperoleh ADC clock 125 kHz, yang menghasilkan waktu konversi sekitar 104 mikrodetik per konversi [4].

Fungsi analogRead() pada Arduino digunakan untuk membaca nilai analog dari pin input. Arduino Uno memiliki 6 pin analog (A0-A5) yang dapat digunakan. Fungsi map() digunakan untuk mengubah (menykalakan) nilai dari satu rentang ke rentang lainnya, misalnya dari rentang ADC (0-1023) ke rentang sudut servo (0-180) atau rentang PWM (0-255).

Rumus konversi ADC ke tegangan adalah sebagai berikut:

di mana ADC_Value adalah nilai yang terbaca dari fungsi analogRead() dan VREF adalah tegangan referensi (default 5V) [4].

Tabel berikut menunjukkan beberapa contoh konversi nilai ADC ke tegangan:

Tabel 2-1 Konversi Nilai ADC ke Tegangan

Nilai ADC	Tegangan (VREF = 5V)	Tegangan (VREF = 3.3V)
0	0.000 V	0.000 V
256	1.250 V	0.825 V
512	2.500 V	1.650 V
768	3.750 V	2.475 V
1023	4.995 V	3.297 V

2.2 Pulse Width Modulation (PWM)

Pulse Width Modulation (PWM) merupakan metode pengaturan sinyal digital dengan cara memodulasi lebar pulsa (duty cycle) dalam satu periode waktu tertentu. Dengan mengubah perbandingan antara waktu sinyal berada pada kondisi HIGH dan LOW, mikrokontroler dapat menghasilkan tegangan rata-rata efektif yang berbeda-beda, sehingga seolah-olah menghasilkan sinyal analog dari keluaran digital [2].

Pada Arduino Uno, fungsi analogWrite() digunakan untuk menghasilkan sinyal PWM pada pin yang mendukung (pin 3, 5, 6, 9, 10, 11, ditandai dengan simbol ~). Frekuensi sinyal PWM pada Arduino Uno adalah sekitar 490 Hz (atau 980 Hz pada pin 5 dan 6). Nilai duty cycle diatur dengan rentang 0-255, di mana 0 berarti 0% (mati) dan 255 berarti 100% (penuh) [3].

Duty cycle didefinisikan sebagai rasio waktu sinyal HIGH terhadap total periode sinyal, dinyatakan dalam persen. Duty cycle 50% berarti sinyal HIGH selama 50% dari periode dan LOW selama 50% sisanya. Tegangan rata-rata yang dihasilkan sebanding dengan duty cycle tersebut. Untuk tegangan 5V, duty cycle 50% menghasilkan tegangan rata-rata 2.5V [5].

Tabel berikut menunjukkan hubungan antara nilai PWM dan duty cycle:

Tabel 2-2 Hubungan Nilai PWM, Duty Cycle, dan Tegangan Rata-rata

Nilai PWM	Duty Cycle	Tegangan Rata-rata (5V)
0	0%	0.00 V
64	25%	1.25 V
128	50%	2.50 V
192	75%	3.75 V
255	100%	5.00 V

2.3 Servo

Servo motor adalah aktuator yang dapat dikendalikan untuk bergerak pada posisi sudut tertentu. Servo motor yang umum digunakan dengan Arduino memiliki rentang gerak 0-180 derajat. Servo dikendalikan menggunakan sinyal PWM dengan lebar pulsa spesifik: pulsa 1 ms untuk posisi 0 derajat, 1.5 ms untuk posisi 90 derajat, dan 2 ms untuk posisi 180 derajat [1].

3. Metodologi

3.1 Alat dan Bahan

Percobaan ini menggunakan perangkat keras dan perangkat lunak sebagai berikut:

Tabel 3-1 Daftar Alat dan Bahan

No	Komponen	Keterangan
1	Arduino Uno	Mikrokontroler utama
2	Potensiometer 10K	Input analog variabel
3	Servo Motor SG90	Aktuator output (Percobaan 1)
4	LED Merah	Indikator output (Percobaan 2)
5	Resistor 220 Ohm	Pembatas arus LED
6	Breadboard	Media perakitan rangkaian
7	Kabel Jumper	Penghubung komponen

3.2 Konfigurasi Pin

3.2.1 Percobaan 1: ADC dengan Servo

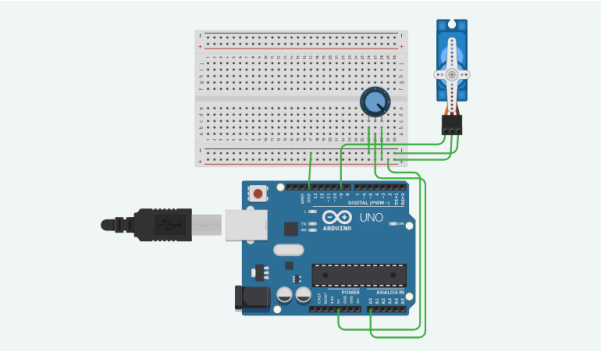
Konfigurasi pin untuk percobaan ADC dengan servo motor adalah sebagai berikut: potensiometer terhubung ke pin analog A0, dengan kaki kiri ke GND, kaki tengah ke A0, dan kaki kanan ke 5V. Servo motor terhubung ke pin digital PWM (misalnya pin 9) untuk sinyal kontrol, dengan kabel daya ke 5V dan GND.

3.2.2 Percobaan 2: PWM dengan LED

Konfigurasi pin untuk percobaan PWM dengan LED adalah sebagai berikut: potensiometer terhubung ke pin analog A0, dengan konfigurasi yang sama seperti percobaan pertama. LED terhubung ke pin digital PWM (misalnya pin 9) melalui resistor 220 Ohm, dengan kaki katoda terhubung ke GND.

3.3 Skema Rangkaian dan Langkah Percobaan

3.3.1 Percobaan 1: ADC dengan Servo



Gambar 3-1 Skema Rangkaian Percobaan 1

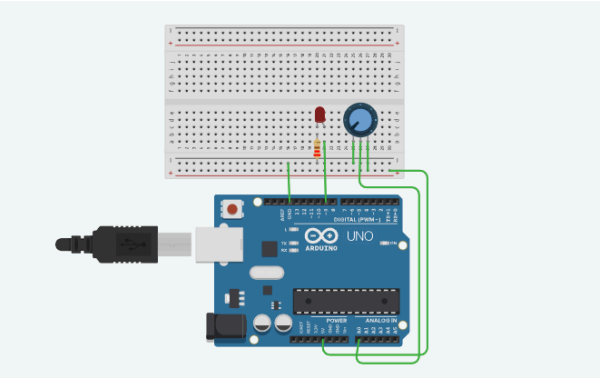
1) Persiapkan alat dan bahan sesuai tabel 3-1

2) Program Percobaan

```
C/C++
#include <Servo.h>
Servo myservo;
const int potensioPin = A0;
const int servoPin = 9;
int pos = 0;
int val = 0;

void setup() {
  myservo.attach(servoPin); // isi dengan
servoPin
  Serial.begin(9600); // isi baud rate
(contoh 9600)

}

void loop() {

  val = analogRead(potensioPin);
  pos = map(val,
            0,
            1023,
            0,
            180);

  myservo.write(pos);
  Serial.print("ADC Potensio: ");
  Serial.print(val);
  Serial.print(" | Sudut Servo: ");
  Serial.println(pos);
  delay(1000);
}
```

3) Simpan sketch dengan nama file modul3_uart
4) Konfigurasi rangkaian sesuai gambar 3-1
5) Berikutnya compile dan upload program ke dalam Arduino board

3.3.2 Percobaan 2: PWM dengan LED



Gambar 3-2 Skema Rangkaian Percobaan 2

1) Persiapkan alat dan bahan sesuai tabel 3-1

2) Program Percobaan

```
C/C++
#include <Arduino.h>
const int potPin = A0;
const int ledPin = 9;
int nilaiADC = 0;
int pwm = 0;

void setup() {
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  nilaiADC = analogRead(potPin);
  pwm = map(nilaiADC,
            0,
            1023,
            0,
            255);

  analogWrite(ledPin, pwm);
  Serial.print("ADC: ");
  Serial.print(nilaiADC);
  Serial.print(" | PWM: ");
  Serial.println(pwm);
  delay(100);
}
```

3) Simpan sketch dengan nama file modul3_i2c
4) Konfigurasi rangkaian sesuai gambar 3-2
5) Berikutnya compile dan upload program ke dalam Arduino board

4. Hasil dan Analisis

4.1 Hasil Percobaan 1: ADC dengan Servo

Pada percobaan pertama, fungsi analogRead() berhasil membaca nilai analog dari potensiometer dengan rentang 0 hingga 1023. Nilai ADC tersebut kemudian

dikonversi ke sudut servo (0-180 derajat) menggunakan fungsi map(). Hasil pengamatan menunjukkan bahwa servo motor bergerak sesuai dengan perubahan posisi potensiometer. Ketika potensiometer diputar ke arah minimum, servo bergerak ke posisi 0 derajat. Ketika potensiometer diputar ke arah maksimum, servo bergerak ke posisi 180 derajat [1].

Serial Monitor berhasil menampilkan data nilai ADC potensiometer dan sudut servo motor secara real-time. Servo tidak mengalami jitter berlebihan selama percobaan, menunjukkan bahwa konversi data dan pengiriman sinyal kontrol berjalan dengan stabil. Delay 15 ms yang ditambahkan pada program memberikan waktu yang cukup bagi servo untuk bergerak secara stabil [3].

Tabel 4-1 Hasil Pengamatan Percobaan 1

Posisi Potensiometer	Nilai ADC	Sudut Servo
Paling kiri (minimum)	0	0 derajat
Kiri-tengah	~256	~45 derajat
Tengah	~512	~90 derajat
Kanan-tengah	~768	~135 derajat
Paling kanan (maksimum)	1023	180 derajat

4.2 Hasil Percobaan 2: PWM dengan LED

Pada percobaan kedua, fungsi analogRead() berhasil membaca nilai analog dari potensiometer dengan rentang 0 hingga 1023. Nilai ADC tersebut kemudian dikonversi ke nilai PWM (0-255) menggunakan fungsi map(). Hasil pengamatan menunjukkan bahwa LED merespons perubahan nilai PWM dengan variasi kecerahan yang sesuai. Ketika nilai potensiometer kecil (ADC rendah), LED menyala redup. Ketika nilai potensiometer besar (ADC tinggi), LED menyala terang [2].

Serial Monitor berhasil menampilkan nilai ADC dan nilai PWM secara real-time. Perubahan kecerahan LED terlihat smooth tanpa flicker yang signifikan, menunjukkan bahwa frekuensi PWM 490 Hz cukup tinggi untuk menghasilkan efek kecerahan yang terlihat kontinu oleh mata manusia [5].

Tabel 4-2 Hasil Pengamatan Percobaan 2

Posisi Potensiometer	Nilai ADC	Kecerahan LED
Paling kiri (minimum)	0	Mati (PWM = 0)
Kiri-tengah	~256	Redup (PWM = 64)
Tengah	~512	Sedang (PWM = 128)
Kanan-tengah	~768	Terang (PWM = 192)
Paling kanan (maksimum)	1023	Penuh (PWM = 255)

4.3 Analisis

Berdasarkan hasil kedua percobaan, dapat dianalisis bahwa modul ADC dan PWM pada Arduino berfungsi dengan baik untuk aplikasi sistem embedded. ADC dengan resolusi 10-bit memberikan presisi yang cukup untuk membaca variasi tegangan dari potensiometer. Nilai ADC 0-1023 memberikan 1024 level diskrit yang memungkinkan kontrol yang halus terhadap perangkat output [4].

Fungsi map() memainkan peran penting dalam menyederhanakan konversi nilai antara rentang yang berbeda. Pada percobaan pertama, map() mengkonversi nilai ADC (0-1023) ke sudut servo (0-180). Pada percobaan kedua, map() mengkonversi nilai ADC (0-1023) ke nilai PWM (0-255). Tanpa fungsi map(), konversi nilai harus dilakukan secara manual dengan perhitungan matematika [1].

Teknik PWM memungkinkan Arduino menghasilkan variasi tegangan rata-rata dari keluaran digital yang secara inheren hanya memiliki dua keadaan (HIGH/LOW). Dengan mengubah duty cycle sinyal PWM, mikrokontroler dapat mengontrol intensitas cahaya LED secara bertahap, mulai dari kondisi redup hingga terang. Hal ini sesuai dengan penelitian yang menunjukkan bahwa PWM merupakan teknik yang efisien untuk kontrol analog pada sistem digital [2].

5. Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa:

- Fungsi analogRead() pada Arduino berhasil digunakan untuk membaca nilai analog dari potensiometer dengan rentang 0-1023 (resolusi 10-bit).
- Fungsi map() berhasil mengkonversi nilai ADC ke sudut servo (0-180 derajat) dan nilai PWM (0-255) dengan akurat.
- Fungsi analogWrite() berhasil menghasilkan sinyal PWM untuk mengontrol kecerahan LED sesuai dengan nilai duty cycle yang diatur.
- Kombinasi ADC dan PWM memungkinkan Arduino untuk membaca input analog dan menghasilkan output analog semu yang responsif untuk aplikasi sistem embedded.

Daftar Pustaka

[1] Kondaveeti, H. K., Kumaravelu, N. K., Vanambathina, S. D., Mathe, S. E., & Vappangi, S. (2021). A Systematic Literature Review on Prototyping with Arduino: Applications, Challenges, Advantages, and Limitations. Computer

Science Review, 40, 100364. https://doi.org/10.1016/j.cosrev.2021.100364

[2] Gobinath, N., & Jeyandra Prabu, S. (2019). Intelligent Lighting System Using Arduino and PWM. International Research Journal of Engineering and Technology (IRJET), 6(12), 2390-2394.

[3] Johnson, D. M., Pate, M., Estepp, C. M., & Wardlow, G. (2023). Using Self-Efficacy Theory to Design Arduino Instruction for Novices: A Replication Study. Journal of Research in Technical Careers, 7(1). https://doi.org/10.9741/2578-2118.1125

[4] Mazidi, M. A., Naimi, S., & Naimi, S. (2017). The AVR Microcontroller and Embedded Systems: Using Assembly and C. MicroDigitalEd.com.

[5] Tselegkaridis, S., Sapounidis, T., & Papakostas, D. (2024). Learning Circuits and Coding with Arduino Board in Higher Education Using Tangible and Graphical User Interfaces. Information, 15(5), 245. https://doi.org/10.3390/info15050245